Emergency Blood Request System

Connect Maratha · Design document · 9 October 2026

1. Overview

When a member needs blood urgently, they raise a request with their location. The request appears on the home page of the app, and compatible donors near the hospital get an immediate push notification. Interested donors tap "I can donate" and call or WhatsApp the requester.

This feature builds on the blood help module that already exists in the Connect Maratha backend (blood requests and donors endpoints) and adds location matching, push notifications, and a request life cycle.

2. Problem and goals

Problem. Today a family in an emergency searches phone contacts and WhatsApp groups one by one. This wastes time, reaches the wrong people, and is hardest for members in villages.

Goals

- Reach compatible donors near the hospital within seconds.
- Show urgent requests on the home page so every member can help or share.
- Keep the exact location and phone number private until a donor responds.
- Keep the feature free for everyone.
- Work for both city and rural members.

Non-goals

- Payments or paid blood.
- Medical screening or storing health records.
- Continuous background tracking of donors.
- Replacing hospitals and blood banks. The app helps find donors and should also point to hospitals and blood banks.

3. Users and roles

- **Requester:** a logged-in member who needs blood for themselves or someone else.
- **Donor:** a member who opted in as a blood donor and is available.
- **Other members:** see the urgent banner and can share the request.
- **Admin:** reviews, closes, or removes requests from the CRM.

4. User flow

1. The requester taps "Emergency blood" and fills in blood group, units needed, hospital, who the patient is (self or other), and a contact name and number.
2. The app takes the current GPS location, or the requester picks the hospital on a map.
3. The backend validates the request, saves it as open, and starts matching.
4. Matched donors get a push notification. All members see an urgent banner on the home page.
5. A donor taps "I can donate". The requester is notified and sees the list of responders.
6. The donor calls or WhatsApps the requester.
7. When enough donors respond, the request moves to "enough donors". The requester marks it fulfilled, or it expires automatically.

5. Features in version 1

- Emergency request form with blood group, units, hospital, urgency, and contact details.
- Home page urgent banner with approximate area and time since posted.
- Donor opt-in and availability toggle in the profile, with last donation date.
- Push notification to matched donors, in Marathi with an English fallback.
- "I can donate" response and requester view of responders.
- Call and WhatsApp buttons for donors.
- Request status: open, enough donors, fulfilled, expired.
- Share request as text or link.

6. System architecture

The system has five layers:

1. **Clients:** the Flutter app (new), the React website, and the admin CRM. All use the same backend.
2. **Nginx:** HTTPS, rate limits, and load balancing. The repo already contains Nginx and PM2 configuration.
3. **Node.js API (Express):** modules for auth, community, blood help, and business. The emergency feature lives in the blood help module.
4. **Data:** PostgreSQL with PostGIS for members, requests, and distance queries; Redis for cache, rate limits, and the job queue; file storage for photos.
5. **Background worker and external services:** a notification worker reads jobs from Redis and calls Firebase Cloud Messaging for push, SMS or WhatsApp as a fallback, and a maps API for geocoding.

The API saves the request and puts a job on the queue, so the requester gets an instant response. The worker then does the slow work of alerting many donors. More workers can be added as traffic grows.

7. Data model

BloodRequest

- `id`, `requesterId`, `forWhom` (self or other), `contactName`, `contactPhone`
- `bloodGroup`, `unitsNeeded`, `unitsPledged`
- `hospital`, `lat`, `lng`, `city`, `district`
- `urgency`, `neededBy`, `status`, `createdAt`, `expiresAt`

DonorProfile

- `memberId`, `bloodGroup`, `optIn`, `available`, `lastDonatedAt`
- `lastKnownLat`, `lastKnownLng`, `lastKnownAt`

DeviceToken

- `memberId`, `fcmToken`, `platform`, `updatedAt`

RequestResponse

- `requestId`, `donorId`, `status` (pledged, contacted, donated, cancelled), `createdAt`

8. API design

- `POST /api/blood/requests`: create a request (login required), triggers matching.

- `GET /api/blood/requests/nearby?lat=&lng=&radius=` : open requests near a point.
- `POST /api/blood/requests/:id/respond` : donor says "I can donate".
- `POST /api/blood/requests/:id/fulfil` : requester closes the request.
- `PUT /api/members/me/donor` : update donor availability, blood group, and last donation date.
- `POST /api/devices` : register the phone's push token.

The existing `GET/POST /api/blood/requests` and `/api/blood/donors` endpoints should be extended rather than replaced.

9. Matching logic

- **Blood compatibility:** for example O negative can give to everyone, and AB positive can receive from everyone.
- **Distance:** start with a radius of about 5 km, widen to 10 and then 25 km if too few donors respond. These steps must be configurable.
- **Eligibility:** donor must be opted in, available, and past the minimum gap since the last donation. The gap differs by gender and country, so make it a setting and confirm it with a doctor or blood bank.
- **Notification fatigue:** limit how often one donor can be alerted, and alert only compatible blood groups.
- **Fallback for version 1:** if GPS is not available, match by district and city.

10. Notification pipeline

1. The API saves the request and adds a job to the Redis queue.
2. The worker finds matching donors and loads their device tokens.
3. Tokens are sent to Firebase Cloud Messaging in batches of 500, with high priority and a dedicated "Emergency" Android channel.
4. An in-app notification is saved for each donor, so it shows even if push fails.
5. Failed or invalid tokens are retried or removed.
6. Example text: "तातडीची रक्त गरज: B+ · पुणे".

11. Privacy and safety

- Ask for location permission with a clear message, and follow the DPDP consent already collected at registration.
- The home page shows only an approximate area (city or district). The exact hospital and phone number are shown only to logged-in donors who respond.
- Donors are not tracked continuously. Use the last known area, updated when they open the app.
- Delete or anonymize the exact location and phone number after a request is fulfilled or expired.
- Contact details are never returned in public list endpoints.
- Show a note that in a life-threatening emergency people should also call 108 and contact the hospital or blood bank.

12. Abuse prevention

- Login and verified membership required to post a request.
- Rate limits per user, and blocking duplicate requests for the same blood group and hospital within a short time.
- A report button on every request.
- Automatic expiry of old requests.
- Admins can close or remove requests and see an audit trail.

13. Admin controls

- View all open and past requests in the admin CRM.
- Close, edit, or remove a request.
- See response counts and fulfilment rate.
- Adjust configuration: radius steps, donor gap in days, request expiry time, alert limits.

14. Reliability and failure handling

- If push fails, the in-app notification and home page banner still show the request.
- Queue jobs are retried with backoff and are safe to repeat without double alerts.

- If the worker is down, requests are still saved and the worker catches up when it restarts.
- Android 13 and above need notification permission, and Android 12 and above let the user share approximate location, which the app must support.
- iOS push requires Apple push credentials configured in Firebase.

15. Current repository versus required changes

Already present: blood requests and donors endpoints, a donor opt-in flag at registration, an in-app notifications list, and Nginx and PM2 configuration.

Missing:

- Latitude and longitude on members and requests.
- Device token storage and push sending.
- Distance queries, request status, and expiry.
- A job queue and notification worker.
- A production database suited to distance queries. The backend currently stores data in a JSON file, so moving to PostgreSQL is recommended before building this feature.
- Security fixes before any public release: remove the hardcoded passwords and the default JWT secret.

16. Delivery phases

1. **Version 1:** district and city matching, device tokens, Firebase push, home page banner, "I can donate" button, request status and expiry.
2. **Version 2:** GPS and radius matching, requester view of responders, admin dashboard.
3. **Version 3:** radius escalation, WhatsApp links, nearby blood banks as a fallback, SMS for members without the app.

17. Benefits

- **Members:** faster help and fewer wasted calls, including in villages.
- **Organization:** builds trust, gives members a reason to install the app, and creates a verified donor network.
- **App:** more engagement through push notifications and a clear difference from other community apps.

The benefit depends on enough members registering as donors in each area, so a donor registration campaign is needed at launch.

18. Revenue note

The emergency feature should stay free and ad-free, because charging for it would damage trust. It supports revenue indirectly by growing the active user base. Revenue ideas belong to other modules, such as business listings, job postings, premium membership, and event sponsorship. The organization's legal structure and payment rules should be checked with a CA or lawyer before charging.

19. Risks

- Too few registered donors in an area, so alerts find no one.
- Fake or misused requests wasting people's time.
- Privacy complaints if location or phone numbers are exposed.
- The JSON file database cannot support location queries and heavy notification traffic.
- Alert fatigue causing donors to turn notifications off.

20. Open questions

- Which production database and hosting will be used?
- Who in the organization approves and moderates requests?
- What donor gap and radius values should be the defaults?
- Is SMS or WhatsApp budget available for fallback alerts?
- Is a blood bank partnership planned?